

mahalanobis_explanation

2026-02-25

Mahalanobis Distance

Let $\mathbf{Y} \sim N_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, as in it is from a multivariate normal distribution. Consider n vectors $\mathbf{y}_1, \dots, \mathbf{y}_n$ vectors from this distribution. The Mahalanobis squared distance for each $\mathbf{y}_1$ is

$$D(\mathbf{y}_i, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (\mathbf{y}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{y}_i - \boldsymbol{\mu}).$$

This is a measure of the distance between a vector and its probability distribution. The idea is to standardize the vector using the mean and deviation. To do this, we need the MLE estimates $\hat{\boldsymbol{\mu}}$ and $\hat{\boldsymbol{\Sigma}}$.

Matrix Variate Normal vs Multivariate Normal

Suppose we are interested in testing if data comes from a matrix variate normal or multivariate normal distribution. Now instead of only one covariance matrix $\boldsymbol{\Sigma}$, we consider two covariance matrices $\mathbf{U}$ and $\mathbf{V}$ that describe the covariance structure of the rows and columns. Additionally, instead of a single mean vector $\boldsymbol{\mu}$, there is a mean matrix $\mathbf{M}$.

We would like to get the Mahalanobis distance under the assumption the data comes from a matrix variate normal distribution.

$$D_M(\mathbf{X}_i, \mathbf{M}, \mathbf{U}, \mathbf{V}) = \text{tr}\{\mathbf{U}^{-1}(\mathbf{X}_i - \mathbf{M})\mathbf{V}^{-1}(\mathbf{X}_i - \mathbf{M})^T\}.$$

This is equivalent to

$$D_M(\mathbf{X}_i, \mathbf{M}, \mathbf{U}, \mathbf{V}) = \text{vec}(\mathbf{X}_i - \mathbf{M})^T (\mathbf{V}^{-1} \otimes \mathbf{U}^{-1}) \text{vec}(\mathbf{X}_i - \mathbf{M}).$$

The idea still being to standardize the original draws, now with the matrix mean $\mathbf{M}$ and with both the covariance matrices $\mathbf{U}$ and $\mathbf{V}$.

Now we would like to define a test for if data is from a matrix variate normal or multivariate normal distribution.

By definition, the matrix variate normal and multivariate normal distributions have a clear relationship.

$$\mathbf{X} \sim N_{r \times c}(\mathbf{M}, \mathbf{U}, \mathbf{V}) \iff \text{vec}(\mathbf{X}) \sim N_{r \times c}(\text{vec}(\mathbf{M}), \mathbf{V} \otimes \mathbf{U}).$$

If the data are matrix variate normal, then a Kroncker product structure does exist. Then $D(\mathbf{y}_i, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = D_M(\mathbf{X}_i, \mathbf{M}, \mathbf{U}, \mathbf{V})$. That is, the Mahalanobis distances for the multivariate normal version and the matrix variate version of the data should be equivalent.

Kolmogorov-Smirnov test

After computing the multivariate normal Mahalanobis distances (denoted as D) and the matrix variate normal Mahalanobis distances (denoted as D_M), we can perform a KS test to see if the distances have the same distribution.

We compute the empirical distributions of the ordered distances using

$$F_N(D) = \frac{1}{N} \sum_{i=1}^N 1_{(-\infty, D]}(D_i).$$

The KS statistic for two samples of cumulative distribution functions is $D_{N,N} = \sup |F_{1,N_a}(D) - F_{2,N_b}(D)|$. That is, the largest distance between the empirical functions of the two distances.

Let H_0 be that D and D_M come from the same distribution, while H_a is they do not come from the same distribution. The null is rejected if

$$D_{N,N} > \sqrt{-\frac{1}{2} \log(\alpha) \left(\frac{2N}{N^2} \right)}.$$

Tensor Variate Normal vs Multivariate Normal

The above equations can be generalized to a tensor variate normal of any order. Instead of only two covariance matrices $\mathbf{U}$ and $\mathbf{V}$, we now have d covariance matrices denoted as $\Sigma_1, \dots, \Sigma_d$. The idea of vectorizing the data and obtaining the Mahalanobis distance under the multivariate normal draws remains the same.

When computing the tensor variate normal distances, we now must use each covariance matrix. Take a 3rd order tensor as an example.

$$D_T(\mathcal{X}_i, \mathcal{M}, \Sigma_1, \Sigma_2, \Sigma_3) = \text{vec}(\mathcal{X}_i - \mathcal{M})^T (\Sigma_1^{-1} \otimes \Sigma_2^{-1} \otimes \Sigma_3^{-1}) \text{vec}(\mathcal{X}_i - \mathcal{M}).$$

Function in tensortools

```
mahalanobis_dist <- function(draws) {
  n <- length(draws) # number of samples
  dim_draws <- dim(draws[[1]])
  num_elem <- prod(dim_draws)
  o <- length(dim_draws)

  # compute tensor Mahalanobis first
  sample_stats <- tensor_mle(draws, model = "normal") # compute MLE estimates

  sample_mean <- sample_stats$mu

  tensor_dist <- rep(0, n)

  sigma_inv <- lapply(sample_stats$sigmas, invert_safe)

  for(i in 1:n) {
    centered_curr <- draws[[i]] - sample_mean # center draws
```

```

centered_multiply <- centered_curr

# standardize with sigma estimates
for(j in 1:o) {
  centered_multiply <- n_prod(centered_multiply, sigma_inv[[j]], j)
}

tensor_dist[i] <- c(centered_curr) %*% c(centered_multiply)
}

# transform to array of n draws
draws_array <- simplify2array(draws) |>
  aperm(c(o+1, 1:o))

# compute vector Mahalanobis
vec_draws <-
  matrix(data = draws_array, nrow = n, ncol = num_elem)

mu_vec <- apply(vec_draws, MARGIN = 2, FUN = mean)

vec_dist <- mahalanobis(vec_draws, center = mu_vec,
  cov = invert_safe(cov(vec_draws)), inverted = TRUE)

# scaling factor is divide by number of elements in a draw
tensor_dist <- tensor_dist * mean(vec_dist) / mean(tensor_dist)

data.frame(vec = vec_dist, tensor = tensor_dist)
}

```

Normal Dist Mahalanobis + KS

Simulate matrix variate data

```

n <- 1e3

s1 <- matrix(rnorm(4), nrow = 2) |> crossprod()
s2 <- matrix(rnorm(9), nrow = 3) |> crossprod()

# error if sigmas don't match mu

# univar normal
univarnorm_draws <- rtnorm(n, mu = 1:2, sigmas = list(s1))
univar_distances <- mahalanobis_dist(univarnorm_draws)

# matrix normal
matnorm_draws <- rtnorm(n, mu = matrix(1:6, nrow = 2),
  sigmas = list(s1, s2))
matnorm_distances <- mahalanobis_dist(matnorm_draws)

# multivariate normal draws

```

```
s_multivar <- matrix(rnorm(6*6), nrow = 6) |> crossprod()
multivar_draws <- rtnorm(n, mu = 1:6, sigmas = list(s_multivar))
```

```
# make a 2x3 array
multivar_draws_3dim <-
  lapply(multivar_draws, function(x) {
    array(x, dim = c(2, 3))
  })
```

```
multivar_distances <- mahalanobis_dist(multivar_draws_3dim)
```

```
univarnorm_test <- mahalanobis_test(univar_distances)
```

```
## Warning in ks.test.default(vec, tensor): p-value will be approximate in the
## presence of ties
```

```
matnorm_test <- mahalanobis_test(matnorm_distances)
multivarnorm_test <- mahalanobis_test(multivar_distances)
```

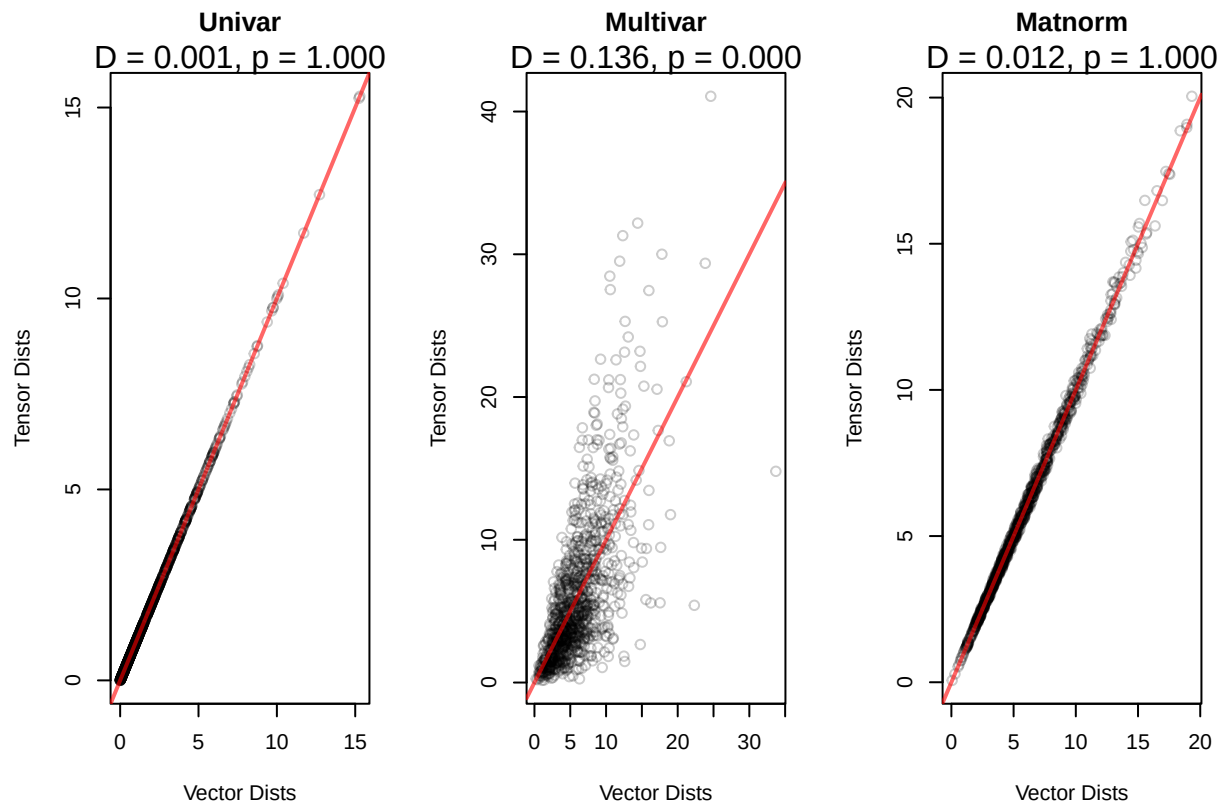
Plot matrix variate results

```
#should this be an object with default plot behavior?
par(mfcol = c(1, 3))

plot_dist(univar_distances)
title(main = "Univar")
with(univarnorm_test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
     side = 3, line = 1)

plot_dist(multivar_distances)
title(main = "Multivar")
with(multivarnorm_test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
     side = 3, line = 1)

plot_dist(matnorm_distances)
title(main = "Matnorm")
with(matnorm_test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
     side = 3, line = 1)
```



Simulate tensor variate data

```
s3 <- matrix(rnorm(16), nrow = 4) |> crossprod()

tensorvarnorm_draws <- rtnorm(n, mu = array(1:(2*3*4), dim = c(2, 3, 4)),
                              sigmas = list(s1, s2, s3))

tensor_distances <- mahalanobis_dist(tensorvarnorm_draws)
tensornorm_test <- mahalanobis_test(tensor_distances)
```

Plot tensor variate results

```
par(mfcol = c(1, 2))

# multivariate normal draws
s_multivar_12 <- matrix(rnorm(12*12), nrow = 12) |> crossprod()
multivar12_draws <- rtnorm(n, mu = 1:12, sigmas = list(s_multivar_12))

# make a 2x3x4 array
multivar12_draws_3dim <-
  lapply(multivar12_draws, function(x) {
    array(x, dim = c(2, 3, 4))
  })
```

```

multivar12_distances <- mahalanobis_dist(multivar12_draws_3dim)

tensornorm_test <- mahalanobis_test(multivar12_distances)

plot_dist(multivar12_distances)
title(main = "Multivar")
with(tensornorm_test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
     side = 3, line = 1)

plot_dist(tensor_distances)
title(main = "Tensor variate")
with(tensornorm_test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
     side = 3, line = 1)

```

